

# Análise de Rede Semântica para Estimação Dimensional e Validação de Conteúdo de Itens sobre Desenvolvimento Musical na Segunda Infância

Frederico Pedrosa

2025-08-06

## Contents

|    |                                                                                        |    |
|----|----------------------------------------------------------------------------------------|----|
| 1  | Introdução                                                                             | 2  |
| 2  | <i>Chamando as bibliotecas e inserindo as frases dos itens</i>                         | 2  |
| 3  | Pré-processamento de itens: retirada de itens ambíguos ou com mais de um comportamento | 2  |
| 4  | Generação de <i>Embeddings</i> para os itens com o Sentence-BERT                       | 3  |
| 5  | Transformação para o formato correto para o pacote EGAnet                              | 4  |
| 6  | Estimação de dimensionalidade e redução de itens iterativamente usando: passo 1        | 4  |
| 7  | Estimação de dimensionalidade e redução de itens iterativamente usando: passo 2        | 6  |
| 8  | Estimação de dimensionalidade e redução de itens iterativamente usando: passo 3        | 10 |
| 9  | ESTIMAÇÃO DE DIMENSIONALIDADE E REDUÇÃO DE ITENS: PASSO 4 (ITERATION 4)                | 13 |
| 10 | Análise “final”                                                                        | 20 |
| 11 | Análise de centralidade                                                                | 23 |
| 12 | Medidas de centralidade e estabilidade                                                 | 25 |
| 13 | Referências                                                                            | 27 |

# 1 Introdução

Esta sintaxe se trata de uma análise de redes semântica de itens de uma escala de desenvolvimento musical para crianças de 3 a 6 anos, baseada na dissertação do musicoterapeuta Antônio Bruno Verga Pinto (2024). Utilizando a arquitetura de transformadores (via `sentence-transformers` em Python) e o framework AIGENIE (Lasalandra et al., 2025), o processo visa:

1. **Refinar o conjunto inicial de itens:** Através de uma análise de validade de conteúdo auxiliada por inteligência artificial, transformando itens textuais em representações semânticas (embeddings) e aplicando técnicas iterativas de redução.
2. **Estimar a estrutura dimensional subjacente:** Empregando a Análise Exploratória de Grafos (EGA e bootEGA do pacote EGAnet) para identificar o número de dimensões empíricas e a alocação dos itens nessas dimensões, após múltiplas etapas de remoção de itens redundantes (via UVA do EGAnet) e instáveis.
3. **Avaliar a importância relativa dos itens:** Conduzindo uma análise de centralidade (`estimateNetwork` e `centralityPlot` dos pacotes `bootnet` e `EGAnet/qgraph`) na rede final de itens para identificar os nós (itens) mais influentes (Força, Proximidade, Intermediação e Influência Esperada) e testando a estabilidade dessas medidas de centralidade (`bootnet` e `corStability`).

As dimensões teóricas originalmente propostas pelos especialistas, que serviram de base para a criação dos itens, foram:

- SENSÓRIO-MOTOR
- SENSÓRIO-MOTOR CONSCIÊNCIA CORPORAL
- SENSÓRIO-MOTOR COM OBJETOS
- SENSÓRIO-MOTOR COM O AMBIENTE
- COMUNICAÇÃO RECEPTIVA
- COMUNICAÇÃO EXPRESSIVA
- HABILIDADES MOTORAS ORAIS
- COGNITIVO ATENÇÃO/PERCEPÇÃO
- COGNITIVO MEMÓRIA/FUNÇÕES EXECUTIVAS
- COGNITIVO HABILIDADES ACADÊMICAS
- VISUAL
- SOCIOEMOCIONAL PARTICIPAÇÃO
- SOCIOEMOCIONAL TOMADA DE TURNO
- SOCIOEMOCIONAL HABILIDADES RELACIONAIS

O objetivo final é obter um conjunto robusto e psicometricamente sólido de itens que representem adequadamente os construtos do desenvolvimento musical na faixa etária especificada, além de identificar os itens com maior impacto na estrutura da rede.

## 2 Chamando as bibliotecas e inserindo as frases dos itens

## 3 Pré-processamento de itens: retirada de itens ambíguos ou com mais de um comportamento

Considerando as boas práticas na redação de itens psicométricos do IPIP, tais como:

##\* Evitar mais de uma habilidade ou ideia por item ##\* Usar linguagem clara, simples e direta ##\* Evitar termos absolutos ou extremos

```

raw_items <- phrases

# Define indices of items to be removed based on prior manual review
# Categories: non-musical, problematic (validity/clarity issues), duplicates/redundant
initial_item_removal_indices <- c(
  # Non-musical content items
  6, 10, 11, 17, 21, 23, 29, 30, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42,
  43, 45, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 66, 70, 87, 88,
  89, 91, 92, 100, 101, 133, 134, 139, 145, 172:174,
  # Problematic items (ambiguity, difficult interpretation, etc.)
  7, 15, 65, 68, 72, 77, 82, 93, 94, 95:116, 124, 131, 137, 138, 149:170,
  # Duplicate or highly redundant items identified manually
  20, 95, 96, 125:128
)

# Remove the specified items
filtered_items <- raw_items[-initial_item_removal_indices]

# Identify and remove any remaining exact duplicate phrases
unique_phrases <- unique(filtered_items)
n_unique_phrases <- length(unique_phrases)

print(paste("Number of unique phrases after initial filtering and deduplication:", n_unique_phrases))

## [1] "Number of unique phrases after initial filtering and deduplication: 64"

```

## 4 Geração de *Embeddings* para os itens com o Sentence-BERT

```

# Specify the dedicated Python virtual environment for sentence-transformers
# Adjust the path to your virtual environment if necessary
use_virtualenv("~/Brysa/.venv_st_ega", required = TRUE)

# Import the sentence-transformers library
# delay_load = FALSE ensures it loads immediately to catch errors early
st_module <- import("sentence_transformers", delay_load = FALSE)
SentenceTransformer <- st_module$SentenceTransformer

# Define the pre-trained sentence embedding model
# 'paraphrase-multilingual-mpnet-base-v2' is suitable for multiple languages
embedding_model_name <- 'paraphrase-multilingual-mpnet-base-v2'

# Load the sentence transformer model
st_model <- SentenceTransformer(embedding_model_name)

# Encode the unique phrases into numerical embeddings
phrase_embeddings_matrix <- st_model$encode(unique_phrases)
print("Embeddings generated.")

## [1] "Embeddings generated."

```

## 5 Transformação para o formato correto para o pacote EGAnet

```
# Following the AI-GENIE paper: treat items as variables (columns) and
# embedding dimensions as observations (rows).

# Transpose the matrix: Dimensions x Items
embeddings_matrix_transposed <- t(phrase_embeddings_matrix)

# Convert to a data frame suitable for EGAnet
embeddings_df_transposed <- as.data.frame(embeddings_matrix_transposed)

# Assign meaningful column names (representing the items)
item_colnames <- paste0("i", 1:n_unique_phrases)
colnames(embeddings_df_transposed) <- item_colnames

# Verify dimensions and structure
print("Dimensions of the data frame for EGA (Embedding Dimensions x Items):")

## [1] "Dimensions of the data frame for EGA (Embedding Dimensions x Items):"

print(dim(embeddings_df_transposed))

## [1] 768 64
```

## 6 Estimação de dimensionalidade e redução de itens iterativamente usando: passo 1

```
# Set seed for reproducibility of bootstrap analysis
set.seed(123)

# --- Iteration 1 ---
print("--- Starting Iteration 1 of UVA and bootEGA ---")

## [1] "--- Starting Iteration 1 of UVA and bootEGA ---"

current_data_iter1 <- embeddings_df_transposed

# Step 3 (AI-GENIE): Unique Variable Analysis to remove redundant items
# Using default settings initially (cutoff might be adjusted based on AI-GENIE paper if needed)
print("Running UVA Iteration 1...")

## [1] "Running UVA Iteration 1..."

uva_results_iter1 <- UVA(data = current_data_iter1, verbose = FALSE)
print(paste("UVA Iteration 1 completed. Items kept:", ncol(uva_results_iter1$reduced_data)))

## [1] "UVA Iteration 1 completed. Items kept: 51"
```

```
# print(uva_results_iter1$keep_remove) # Uncomment to see details
```

```
# Store data after UVA reduction
```

```
data_after_uva_iter1 <- uva_results_iter1$reduced_data
```

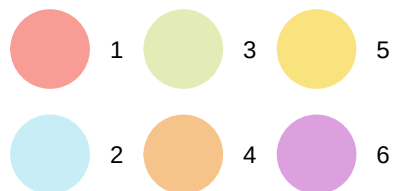
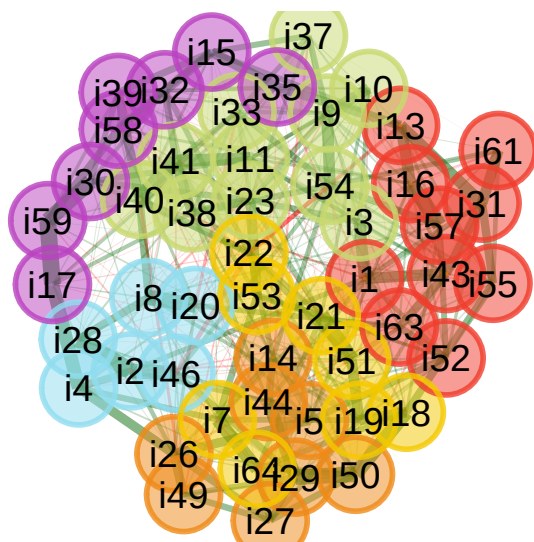
```
# Step 5 (AI-GENIE): Bootstrap EGA to assess item stability
```

```
print("Running bootEGA Iteration 1...")
```

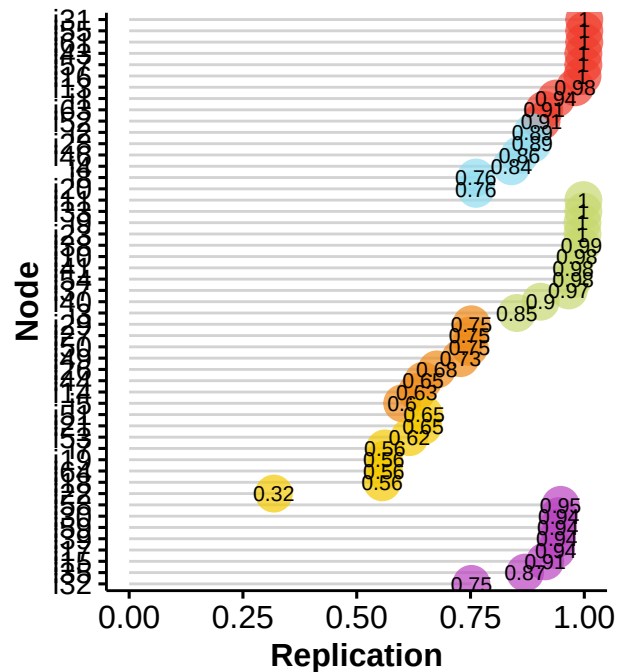
```
## [1] "Running bootEGA Iteration 1..."
```

```
bootEGA_results_iter1 <- bootEGA(
  data = data_after_uva_iter1,
  iter = 500, # Standard number of iterations, adjust if needed
  type = "parametric", # Or "resampling"
  model = "glasso", # As used later, keep consistent
  seed = 123 # Ensure reproducibility within bootEGA too
)
```

### Original Sample | EGA



### Communities



```
print("bootEGA Iteration 1 completed.")
```

```
## [1] "bootEGA Iteration 1 completed."
```

```

# Extract item stability proportions (empirical dimension stability)
item_stability_proportions_iter1 <- bootEGA_results_iter1$stability$item.stability$item.stability$empir

# Define the stability threshold (as per AI-GENIE paper or standard practice)
stability_threshold <- 0.75

# Identify stable and unstable items
stable_items_iter1 <- names(item_stability_proportions_iter1[item_stability_proportions_iter1 >= stability_threshold])
unstable_items_iter1 <- names(item_stability_proportions_iter1[item_stability_proportions_iter1 < stability_threshold])

print(paste("Number of stable items (>= 0.75) after Iteration 1:", length(stable_items_iter1)))

## [1] "Number of stable items (>= 0.75) after Iteration 1: 36"

print(paste("Number of unstable items (< 0.75) after Iteration 1:", length(unstable_items_iter1)))

## [1] "Number of unstable items (< 0.75) after Iteration 1: 15"

if (length(unstable_items_iter1) > 0) {
  print("Unstable items to be removed for next iteration:")
  print(unstable_items_iter1)
}

## [1] "Unstable items to be removed for next iteration:"
## [1] "i5" "i7" "i14" "i18" "i19" "i21" "i22" "i26" "i27" "i44" "i49" "i50"
## [13] "i51" "i53" "i64"

# Create the dataset for the next iteration, keeping only stable items
if (length(stable_items_iter1) > 0) {
  stable_items_data_iter1 <- data_after_uva_iter1[, stable_items_iter1, drop = FALSE]
  print("Dimensions of stable data after Iteration 1 (Dimensions x Stable Items):")
  print(dim(stable_items_data_iter1))
} else {
  stop("No stable items found after Iteration 1. Stopping.")
}

## [1] "Dimensions of stable data after Iteration 1 (Dimensions x Stable Items):"
## [1] 768 36

```

Os itens que tem alta redundância semântica ou que não são estáveis nas replicações dentro de suas comunidades são excluídos para a próxima fase.

## 7 Estimação de dimensionalidade e redução de itens iterativamente usando: passo 2

```

# --- Iteration 2 ---
# Bloco if/else que decide se a Iteração 2 é necessária
if (length(unstable_items_iter1) == 0) {
  print("---- No unstable items removed in Iteration 1. Using Iteration 1 results as final. ----")
  final_stable_data <- stable_items_data_iter1
  # Rodar um EGA final se houver dados suficientes
  if (!is.null(final_stable_data) && ncol(final_stable_data) >= 2) {
    print("Running final EGA on stable data from Iteration 1...")
    final_ega_run <- EGA(data = final_stable_data, model = "glasso", verbose = FALSE)
  } else if (!is.null(final_stable_data) && ncol(final_stable_data) > 0) {
    print("Final stable data from Iteration 1 has less than 2 items. Skipping final EGA.")
    # Você pode querer atribuir algo a final_ega_run ou lidar com isso na seção de relatório
    final_ega_run <- list(n.dim = 0, wc = NULL) # Exemplo de placeholder
  } else {
    print("No stable data from Iteration 1. Stopping or handling as error.")
    # stop("No stable data to proceed with after Iteration 1.")
    final_stable_data <- data.frame() # Para evitar erros no código subsequente se ele esperar final_st
    final_ega_run <- list(n.dim = 0, wc = NULL)
  }
} else {
  print("---- Starting Iteration 2 of UVA and bootEGA ----")
  current_data_iter2 <- stable_items_data_iter1

  # Diagnóstico: verificar dimensões antes do UVA da Iteração 2
  print(paste("Dimensions of data for UVA Iteration 2 (current_data_iter2):",
    nrow(current_data_iter2), "rows x", ncol(current_data_iter2), "cols"))

  # Se houver menos de 2 itens, UVA pode não ser apropriado ou bootEGA falhará de qualquer forma
  if (ncol(current_data_iter2) < 2) {
    print("Less than 2 items entering Iteration 2. Using current data as 'data_after_uva_iter2'.")
    data_after_uva_iter2 <- current_data_iter2
    # Se ainda for < 2, bootEGA falhará, então podemos parar ou pular bootEGA
    if (ncol(data_after_uva_iter2) < 2) {
      print("Still less than 2 items after deciding to skip UVA. Skipping bootEGA for Iteration 2.")
      # Se for este o caso, os itens estáveis da iteração 2 serão os mesmos que entraram
      stable_items_data_iter2 <- data_after_uva_iter2
      unstable_items_iter2 <- character(0) # Nenhum item instável se pulamos o bootEGA
      # Define final_stable_data e final_ega_run aqui, pois não haverá Iteração 3
      final_stable_data <- stable_items_data_iter2
      if (!is.null(final_stable_data) && ncol(final_stable_data) >= 2) {
        print("Running final EGA on data from Iteration 2 (skipped bootEGA)...")
        final_ega_run <- EGA(data = final_stable_data, model = "glasso", verbose = FALSE)
      } else {
        final_ega_run <- list(n.dim = 0, wc = NULL) # Placeholder
      }
      # Para pular a Iteração 3, precisamos garantir que unstable_items_iter2 seja vazio
      # O código da Iteração 3 já verifica isso.
    }
  } else {
    print("Running UVA Iteration 2...")
    uva_results_iter2 <- UVA(data = current_data_iter2, verbose = FALSE)

    # CORREÇÃO PRINCIPAL AQUI:

```

```

# Verificar se uva_results_iter2$reduced_data é NULL ou tem 0 colunas
if (is.null(uva_results_iter2$reduced_data) || ncol(uva_results_iter2$reduced_data) == 0) {
  print("UVA Iteration 2 did not reduce items further or resulted in empty data. Using input data")
  data_after_uva_iter2 <- current_data_iter2
} else {
  data_after_uva_iter2 <- uva_results_iter2$reduced_data
}
print(paste("UVA Iteration 2 completed. Items now in 'data_after_uva_iter2':", ncol(data_after_uva_iter2)))
}

# Apenas prossiga com bootEGA se data_after_uva_iter2 tiver itens suficientes
# E se não pulamos o bootEGA anteriormente (ou seja, se ncol(current_data_iter2) >= 2 inicialmente)
if (exists("data_after_uva_iter2") && !is.null(data_after_uva_iter2) && ncol(data_after_uva_iter2) >= 2) {
  print("Running bootEGA Iteration 2...")
  bootEGA_results_iter2 <- bootEGA(
    data = data_after_uva_iter2,
    iter = 500, type = "parametric", model = "glasso", seed = 123
  )
  print("bootEGA Iteration 2 completed.")

  item_stability_proportions_iter2 <- bootEGA_results_iter2$stability$item.stability$item.stability
  stable_items_iter2 <- names(item_stability_proportions_iter2[item_stability_proportions_iter2 >= stability_threshold])
  unstable_items_iter2 <- names(item_stability_proportions_iter2[item_stability_proportions_iter2 < stability_threshold])

  print(paste("Number of stable items (>= ", stability_threshold, ") after Iteration 2:", length(stable_items_iter2)))
  print(paste("Number of unstable items (< ", stability_threshold, ") after Iteration 2:", length(unstable_items_iter2)))
  if (length(unstable_items_iter2) > 0) {
    print("Unstable items to be removed for next iteration:")
    # print(unstable_items_iter2) # Descomentar para ver
  }

  if (length(stable_items_iter2) > 0) {
    # Garantir que os itens estáveis realmente existam em data_after_uva_iter2
    valid_stable_items_iter2 <- intersect(stable_items_iter2, colnames(data_after_uva_iter2))
    if (length(valid_stable_items_iter2) != length(stable_items_iter2)) {
      print("Warning: Some stable items from bootEGA Iteration 2 were not found in data_after_uva_iter2")
    }
    if (length(valid_stable_items_iter2) > 0) {
      stable_items_data_iter2 <- data_after_uva_iter2[, valid_stable_items_iter2, drop = FALSE]
      print("Dimensions of stable data after Iteration 2 (Dimensions x Stable Items):")
      print(dim(stable_items_data_iter2))
    } else {
      print("No valid stable items found after Iteration 2 filtering. Setting stable_items_data_iter2 to empty data frame")
      stable_items_data_iter2 <- data.frame()
      # Se não há itens estáveis, não haverá itens instáveis para a próxima iteração
      unstable_items_iter2 <- character(0)
    }
  } else {
    print("No stable items found after Iteration 2. Stopping or setting stable_items_data_iter2 to empty data frame")
    stable_items_data_iter2 <- data.frame() # Ou stop() como você tinha
    # Se não há itens estáveis, não haverá itens instáveis para a próxima iteração
    unstable_items_iter2 <- character(0)
  }
}

```



```

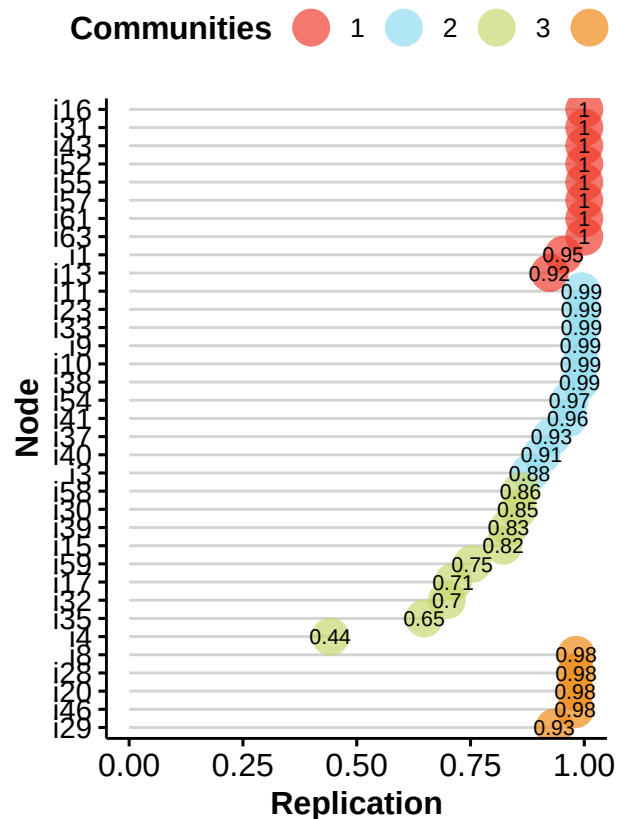
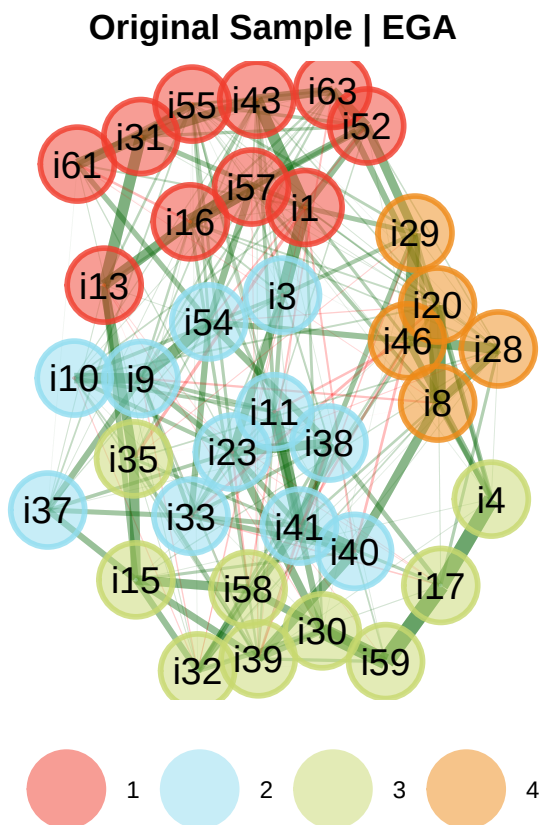
} else if (exists("data_after_uva_iter2") && !is.null(data_after_uva_iter2) && ncol(data_after_uva_iter2) > 0) {
  # Caso em que data_after_uva_iter2 tem 1 item, bootEGA foi pulado.
  print("bootEGA Iteration 2 was skipped as there was only 1 item after UVA.")
  stable_items_data_iter2 <- data_after_uva_iter2 # O único item é considerado "estável" neste contexto
  unstable_items_iter2 <- character(0) # Nenhum instável
  # final_stable_data e final_ega_run já foram definidos no bloco if ncol(current_data_iter2) < 2
} else {
  print("No data to process for bootEGA Iteration 2 or data became empty.")
  stable_items_data_iter2 <- data.frame()
  unstable_items_iter2 <- character(0)
  # Defina final_stable_data e final_ega_run aqui também para o caso de não haver Iteração 3
  final_stable_data <- stable_items_data_iter2
  final_ega_run <- list(n.dim = 0, wc = NULL)
}
}

```

```

## [1] "--- Starting Iteration 2 of UVA and bootEGA ---"
## [1] "Dimensions of data for UVA Iteration 2 (current_data_iter2): 768 rows x 36 cols"
## [1] "Running UVA Iteration 2..."
## [1] "UVA Iteration 2 completed. Items now in 'data_after_uva_iter2': 35"
## [1] "Running bootEGA Iteration 2..."

```



```

## [1] "bootEGA Iteration 2 completed."
## [1] "Number of stable items (>= 0.75) after Iteration 2: 31"

```

```
## [1] "Number of unstable items (< 0.75 ) after Iteration 2: 4"
## [1] "Unstable items to be removed for next iteration:"
## [1] "Dimensions of stable data after Iteration 2 (Dimensions x Stable Items):"
## [1] 768 31
```

Já não existem mais itens redundantes semanticamente, no entanto, alguns itens ainda apresentam baixa comunalidade.

## 8 Estimação de dimensionalidade e redução de itens iterativamente usando: passo 3

```
# =====
# ESTIMAÇÃO DE DIMENSIONALIDADE E REDUÇÃO DE ITENS: PASSO 3 (ITERATION 3)
# =====

# Checagens iniciais para garantir que os objetos da Iteração 2 existam
if (!exists("stable_items_data_iter2")) {
  stable_items_data_iter2 <- data.frame()
}
if (!exists("unstable_items_iter2")) {
  if (length(unstable_items_iter1) == 0) {
    unstable_items_iter2 <- character(0)
  } else {
    # Esta mensagem pode ser removida se o fluxo estiver correto
    # print("Warning: unstable_items_iter2 was not defined before Iteration 3. Assuming no unstable
    unstable_items_iter2 <- character(0)
  }
}

# --- Início da Iteração 3 ---

# Decide se a Iteração 3 é necessária
if (length(unstable_items_iter2) == 0) {
  print("---- No unstable items removed in Iteration 2. Using Iteration 2 results as final. ----")

  # Se paramos aqui, o conjunto de dados da Iteração 3 é o mesmo que o da 2
  stable_items_data_iter3 <- stable_items_data_iter2
  unstable_items_iter3 <- character(0) # Não há instáveis

  # Definindo os objetos finais
  final_stable_data <- stable_items_data_iter3

  if (!is.null(final_stable_data) && ncol(final_stable_data) >= 2) {
    print("Running final EGA on stable data from Iteration 2...")
    final_ega_run <- EGA(data = final_stable_data, model = "glasso", verbose = FALSE)
  } else {
    final_ega_run <- list(n.dim = 0, wc = NULL)
  }
} else { # Bloco que executa a Iteração 3
  print("---- Starting Iteration 3 of UVA and bootEGA ----")
```

```

current_data_iter3 <- stable_items_data_iter2

print(paste("Dimensions of data for UVA Iteration 3:",
            nrow(current_data_iter3), "rows x", ncol(current_data_iter3), "cols"))

# Checagem de segurança
if (ncol(current_data_iter3) < 2) {
  print("Less than 2 items entering Iteration 3. Stopping this iteration.")
  data_after_uva_iter3 <- current_data_iter3
  unstable_items_iter3 <- character(0)
  stable_items_data_iter3 <- current_data_iter3 # O que sobrou é o resultado
} else {
  # UVA
  print("Running UVA Iteration 3...")
  uva_results_iter3 <- UVA(data = current_data_iter3, verbose = FALSE)

  if (is.null(uva_results_iter3$reduced_data) || ncol(uva_results_iter3$reduced_data) == 0) {
    data_after_uva_iter3 <- current_data_iter3
  } else {
    data_after_uva_iter3 <- uva_results_iter3$reduced_data
  }
  print(paste("UVA Iteration 3 completed. Items remaining:", ncol(data_after_uva_iter3)))
}

# bootEGA
if (exists("data_after_uva_iter3") && !is.null(data_after_uva_iter3) && ncol(data_after_uva_iter3) > 0) {
  print("Running bootEGA Iteration 3...")
  bootEGA_results_iter3 <- bootEGA(
    data = data_after_uva_iter3,
    iter = 500, type = "parametric", model = "lasso", seed = 123
  )
  print("bootEGA Iteration 3 completed.")

  item_stability_proportions_iter3 <- bootEGA_results_iter3$stability$item.stability$item.stability
  stable_items_iter3 <- names(item_stability_proportions_iter3[item_stability_proportions_iter3 >= stability_threshold])
  unstable_items_iter3 <- names(item_stability_proportions_iter3[item_stability_proportions_iter3 < stability_threshold])

  print(paste("Number of stable items (>= ", stability_threshold, ") after Iteration 3:", length(stable_items_iter3)))
  print(paste("Number of unstable items (< ", stability_threshold, ") after Iteration 3:", length(unstable_items_iter3)))

  # **** ADIÇÃO PRINCIPAL AQUI ****
  # Cria o dataframe com os itens estáveis desta iteração
  if (length(stable_items_iter3) > 0) {
    stable_items_data_iter3 <- data_after_uva_iter3[, stable_items_iter3, drop = FALSE]
    print("Created 'stable_items_data_iter3' for the next iteration.")
    print(dim(stable_items_data_iter3))
  } else {
    print("No stable items found after Iteration 3. 'stable_items_data_iter3' will be empty.")
    stable_items_data_iter3 <- data.frame()
  }
} else {
  # Se bootEGA foi pulado, os dados estáveis são o que sobrou do UVA

```

```

    stable_items_data_iter3 <- data_after_uva_iter3
    unstable_items_iter3 <- character(0) # Nenhum item instável foi identificado
    print("bootEGA Iteration 3 was skipped. Using data post-UVA as stable data.")
  }

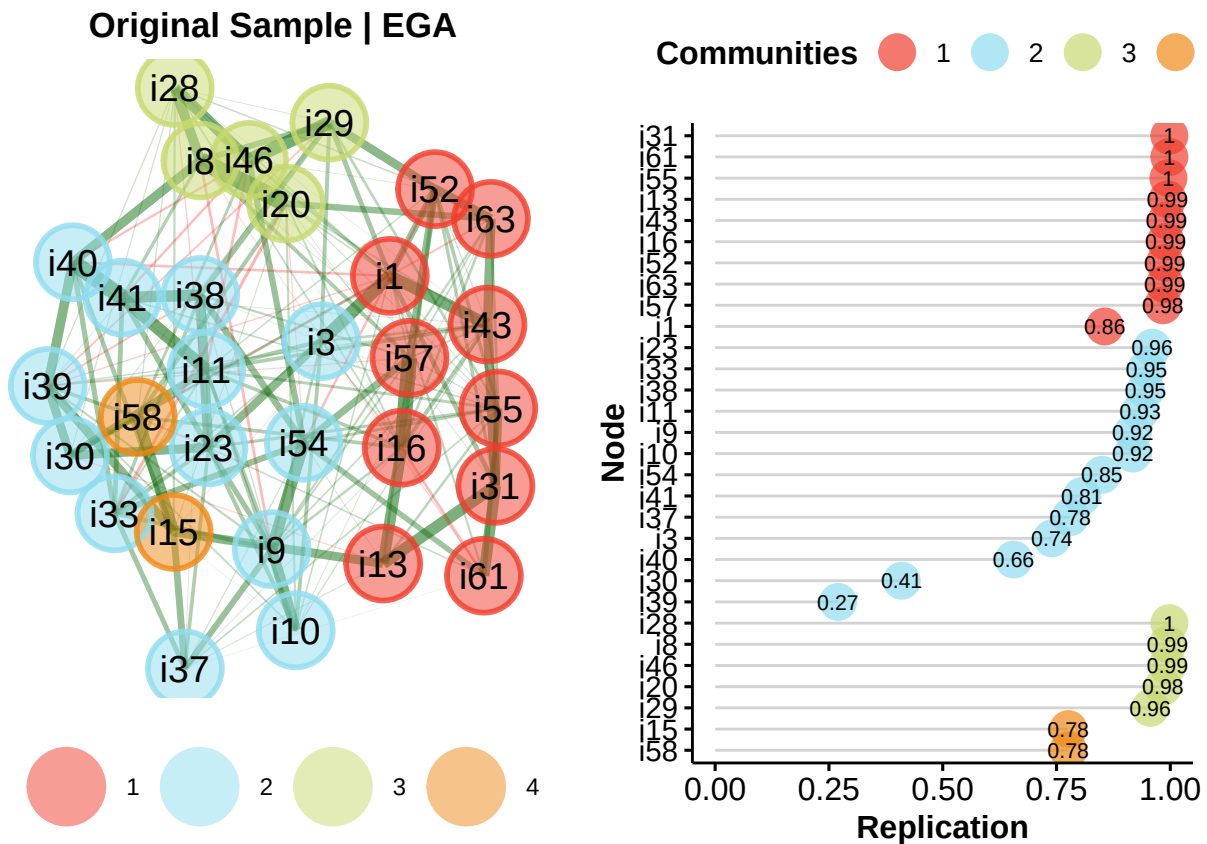
  # O código abaixo, que define final_stable_data e final_ega_run,
  # pode ser removido daqui e colocado apenas no final do fluxo total (após a última iteração),
  # mas o deixarei para manter a consistência com sua estrutura original.
  # Ele basicamente define o resultado "provisório" ao final da rodada 3.
  final_stable_data <- stable_items_data_iter3
}

```

```

## [1] "--- Starting Iteration 3 of UVA and bootEGA ---"
## [1] "Dimensions of data for UVA Iteration 3: 768 rows x 31 cols"
## [1] "Running UVA Iteration 3..."
## [1] "UVA Iteration 3 completed. Items remaining: 30"
## [1] "Running bootEGA Iteration 3..."

```



```

## [1] "bootEGA Iteration 3 completed."
## [1] "Number of stable items (>= 0.75 ) after Iteration 3: 26"
## [1] "Number of unstable items (< 0.75 ) after Iteration 3: 4"
## [1] "Created 'stable_items_data_iter3' for the next iteration."
## [1] 768 26

```

## 9 ESTIMAÇÃO DE DIMENSIONALIDADE E REDUÇÃO DE ITENS: PASSO 4 (ITERATION 4)

```
# Checagens iniciais para garantir que os objetos da Iteração 3 existam
if (!exists("stable_items_data_iter3")) {
  stable_items_data_iter3 <- data.frame() # Cria um dataframe vazio se não existir
}
if (!exists("unstable_items_iter3")) {
  # Assume que se o objeto não foi criado, é porque não havia itens instáveis na rodada anterior
  unstable_items_iter3 <- character(0)
}

# --- Início da Iteração 4 ---

# Bloco if/else que decide se a Iteração 4 é necessária
# Ela só é necessária se houveram itens instáveis na Iteração 3
if (length(unstable_items_iter3) == 0) {
  print("--- No unstable items removed in Iteration 3. Using Iteration 3 results as final. ---")

  # Os dados finais são os estáveis da Iteração 3
  final_stable_data <- stable_items_data_iter3

  # Roda a EGA final, se houver dados suficientes
  if (!is.null(final_stable_data) && ncol(final_stable_data) >= 2) {
    print("Running final EGA on stable data from Iteration 3...")
    final_ega_run <- EGA(data = final_stable_data, model = "glasso", verbose = FALSE)
  } else {
    print("Final stable data from Iteration 3 has less than 2 items. Skipping final EGA.")
    final_ega_run <- list(n.dim = 0, wc = NULL)
  }
} else { # Este bloco executa a Iteração 4
  print("--- Starting Iteration 4 of UVA and bootEGA ---")

  # Os dados de entrada para esta iteração são os itens estáveis da anterior
  current_data_iter4 <- stable_items_data_iter3

  # Diagnóstico: verificar dimensões antes do UVA da Iteração 4
  print(paste("Dimensions of data for UVA Iteration 4 (current_data_iter4):",
    nrow(current_data_iter4), "rows x", ncol(current_data_iter4), "cols"))

  # Checagem de segurança: pular UVA/bootEGA se tivermos poucos itens
  if (ncol(current_data_iter4) < 2) {
    print("Less than 2 items entering Iteration 4. Finalizing with current data.")
    final_stable_data <- current_data_iter4
    final_ega_run <- list(n.dim = ifelse(ncol(final_stable_data) > 0, 1, 0), wc = NULL) # Placeholder
  } else {
    # --- Passo de UVA na Iteração 4 ---
    print("Running UVA Iteration 4...")
    uva_results_iter4 <- UVA(data = current_data_iter4, verbose = FALSE)

    # Se UVA não reduzir ou retornar nulo, usamos os dados de entrada
  }
}
```

```

if (is.null(uva_results_iter4$reduced_data) || ncol(uva_results_iter4$reduced_data) == 0) {
  print("UVA Iteration 4 did not reduce items or resulted in empty data. Using input data.")
  data_after_uva_iter4 <- current_data_iter4
} else {
  data_after_uva_iter4 <- uva_results_iter4$reduced_data
}
print(paste("UVA Iteration 4 completed. Items remaining:", ncol(data_after_uva_iter4)))

# --- Passo de bootEGA na Iteração 4 ---
if (ncol(data_after_uva_iter4) >= 2) {
  print("Running bootEGA Iteration 4...")
  bootEGA_results_iter4 <- bootEGA(
    data = data_after_uva_iter4,
    iter = 500, type = "parametric", model = "glasso", seed = 123
  )
  print("bootEGA Iteration 4 completed.")

  item_stability_proportions_iter4 <- bootEGA_results_iter4$stability$item.stability$item.stability
  stable_items_iter4 <- names(item_stability_proportions_iter4[item_stability_proportions_iter4 >= 0.5])
  unstable_items_iter4 <- names(item_stability_proportions_iter4[item_stability_proportions_iter4 < 0.5])

  print(paste("Number of stable items (>= ", stability_threshold, ") after Iteration 4:", length(stable_items_iter4)))
  print(paste("Number of unstable items (< ", stability_threshold, ") after Iteration 4:", length(unstable_items_iter4)))

  if (length(unstable_items_iter4) > 0) {
    warning("Items STILL remain unstable after 4 iterations. It is recommended to stop and restart.")
  }

  # Os itens estáveis desta rodada se tornam o conjunto de dados final
  if (length(stable_items_iter4) > 0) {
    final_stable_data <- data_after_uva_iter4[, stable_items_iter4, drop = FALSE]
    print("Dimensions of final stable data after Iteration 4:")
    print(dim(final_stable_data))

    # Roda a EGA final sobre o conjunto final de dados
    if (ncol(final_stable_data) >= 2) {
      print("Running final EGA on data from Iteration 4...")
      final_ega_run <- EGA(data = final_stable_data, model = "glasso", verbose = FALSE)
    } else {
      final_ega_run <- list(n.dim = 0, wc = NULL)
    }
  } else {
    print("No stable items found after Iteration 4. Final dataset is empty.")
    final_stable_data <- data.frame()
    final_ega_run <- list(n.dim = 0, wc = NULL)
  }
} else {
  print("bootEGA Iteration 4 was skipped as there were too few items after UVA.")
  final_stable_data <- data_after_uva_iter4
  if(ncol(final_stable_data) > 0){
    final_ega_run <- list(n.dim = 1, wc = NULL) # Tem 1 item, logo 1 dimensão
  } else {
    final_ega_run <- list(n.dim = 0, wc = NULL)
  }
}

```

```

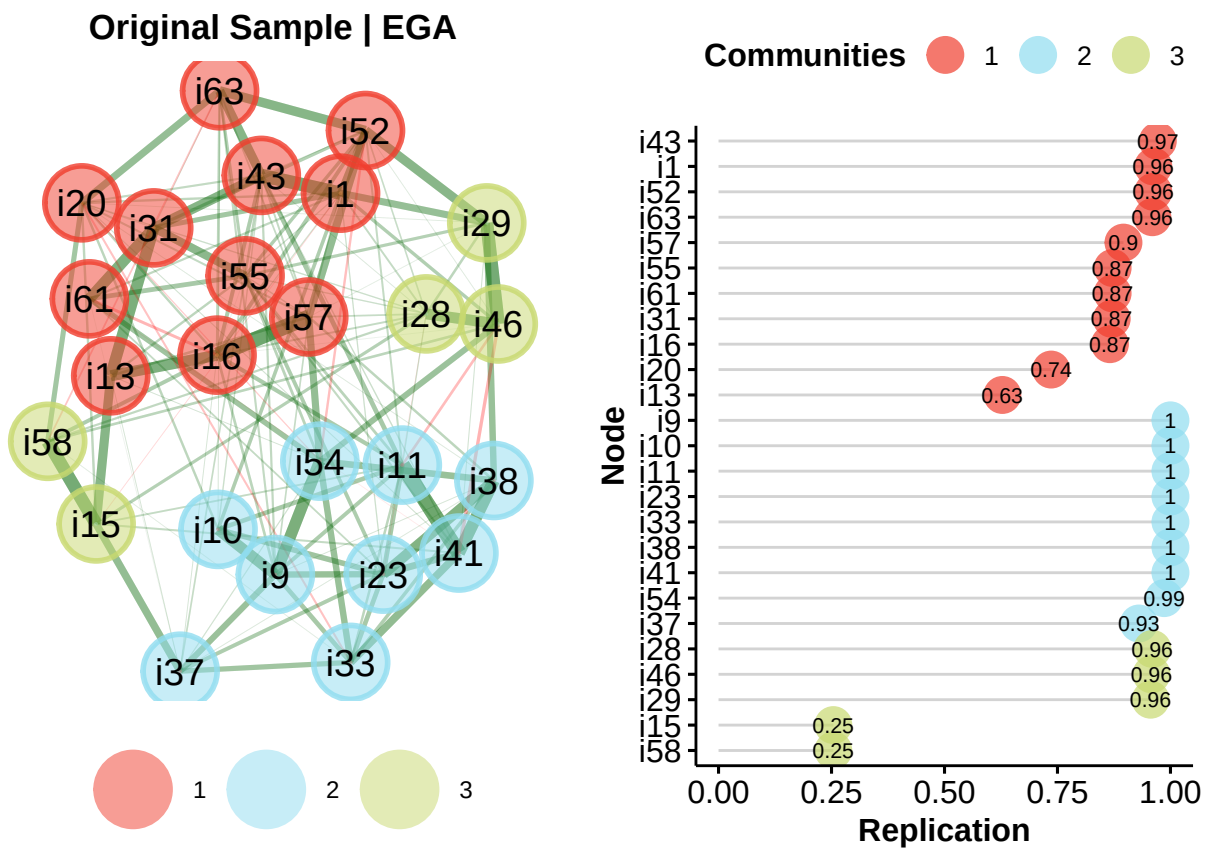
    }
  }
}

```

```

## [1] "--- Starting Iteration 4 of UVA and bootEGA ---"
## [1] "Dimensions of data for UVA Iteration 4 (current_data_iter4): 768 rows x 26 cols"
## [1] "Running UVA Iteration 4..."
## [1] "UVA Iteration 4 completed. Items remaining: 25"
## [1] "Running bootEGA Iteration 4..."

```



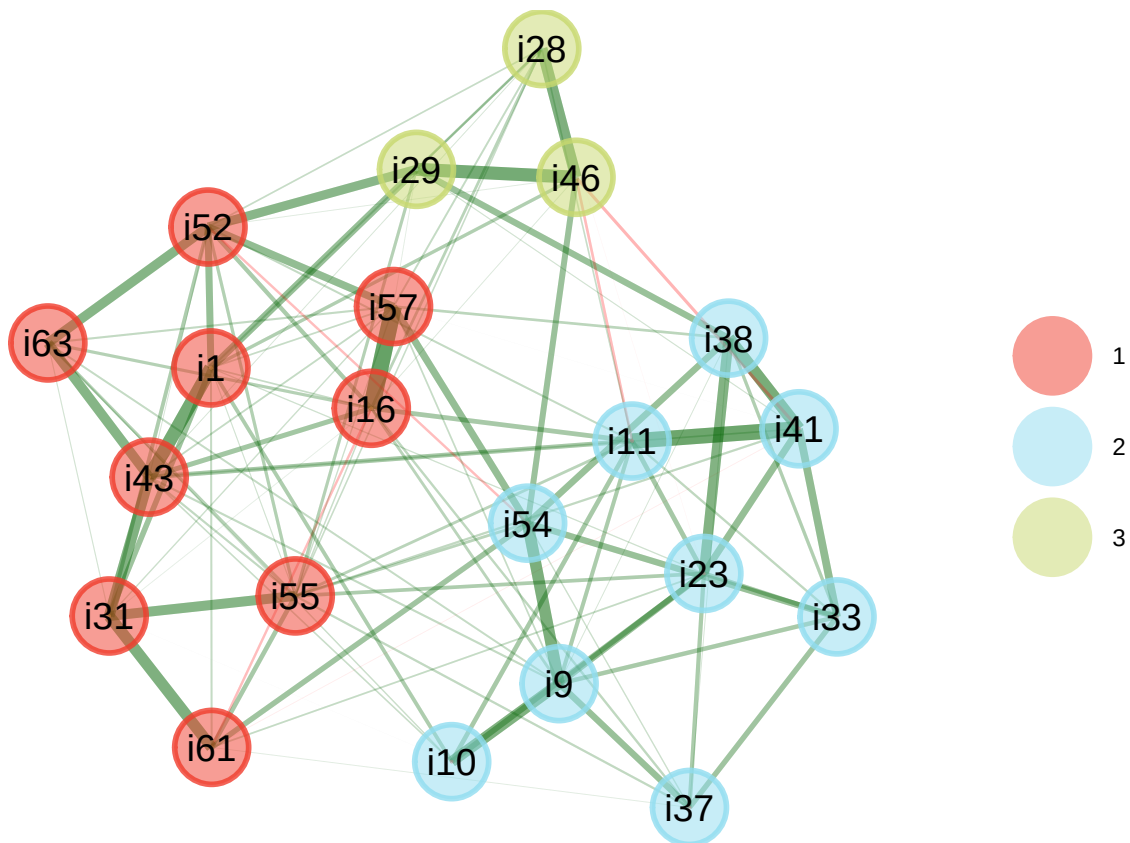
```

## [1] "bootEGA Iteration 4 completed."
## [1] "Number of stable items (>= 0.75 ) after Iteration 4: 21"
## [1] "Number of unstable items (< 0.75 ) after Iteration 4: 4"

## [1] "Dimensions of final stable data after Iteration 4:"
## [1] 768 21
## [1] "Running final EGA on data from Iteration 4..."

```





```
stable_items_data_iter4 <- final_stable_data

# Checagens iniciais para garantir que os objetos da Iteração 4 existam
if (!exists("stable_items_data_iter4")) {
  stable_items_data_iter4 <- data.frame() # Cria um dataframe vazio se não existir
}
if (!exists("unstable_items_iter4")) {
  # Assume que se o objeto não foi criado, é porque não havia itens instáveis na rodada anterior
  unstable_items_iter4 <- character(0)
}

# --- Início da Iteração 5 ---

# Bloco if/else que decide se a Iteração 5 é necessária
# Ela só é necessária se houveram itens instáveis na Iteração 4
if (length(unstable_items_iter4) == 0) {
  print("--- No unstable items removed in Iteration 4. Using Iteration 4 results as final. ---")

  # Os dados finais são os estáveis da Iteração 4
  final_stable_data <- stable_items_data_iter4

  # Roda a EGA final, se houver dados suficientes
  if (!is.null(final_stable_data) && ncol(final_stable_data) >= 2) {
    print("Running final EGA on stable data from Iteration 4...")
    final_ega_run <- EGA(data = final_stable_data, model = "glasso", verbose = FALSE)
  } else {
```



```

    print("Final stable data from Iteration 4 has less than 2 items. Skipping final EGA.")
    final_ega_run <- list(n.dim = 0, wc = NULL)
  }

} else { # Bloco que executa a Iteração 5
  print("---- Starting Iteration 5 of UVA and bootEGA ----")

  # Os dados de entrada para esta iteração são os itens estáveis da anterior
  current_data_iter5 <- stable_items_data_iter4

  # Diagnóstico: verificar dimensões antes do UVA da Iteração 5
  print(paste("Dimensions of data for UVA Iteration 5 (current_data_iter5):",
              nrow(current_data_iter5), "rows x", ncol(current_data_iter5), "cols"))

  # Checagem de segurança: pular UVA/bootEGA se tivermos poucos itens
  if (ncol(current_data_iter5) < 2) {
    print("Less than 2 items entering Iteration 5. Finalizing with current data.")
    final_stable_data <- current_data_iter5
    final_ega_run <- list(n.dim = ifelse(ncol(final_stable_data) > 0, 1, 0), wc = NULL) # Placeholder
  } else {
    # --- Passo de UVA na Iteração 5 ---
    print("Running UVA Iteration 5...")
    uva_results_iter5 <- UVA(data = current_data_iter5, verbose = FALSE)

    # Se UVA não reduzir ou retornar nulo, usamos os dados de entrada
    if (is.null(uva_results_iter5$reduced_data) || ncol(uva_results_iter5$reduced_data) == 0) {
      print("UVA Iteration 5 did not reduce items or resulted in empty data. Using input data.")
      data_after_uva_iter5 <- current_data_iter5
    } else {
      data_after_uva_iter5 <- uva_results_iter5$reduced_data
    }
    print(paste("UVA Iteration 5 completed. Items remaining:", ncol(data_after_uva_iter5)))

    # --- Passo de bootEGA na Iteração 5 ---
    if (ncol(data_after_uva_iter5) >= 2) {
      print("Running bootEGA Iteration 5...")
      bootEGA_results_iter5 <- bootEGA(
        data = data_after_uva_iter5,
        iter = 500, type = "parametric", model = "glasso", seed = 123
      )
      print("bootEGA Iteration 5 completed.")

      item_stability_proportions_iter5 <- bootEGA_results_iter5$stability$item.stability$item.stability
      stable_items_iter5 <- names(item_stability_proportions_iter5[item_stability_proportions_iter5 >= stability_threshold])
      unstable_items_iter5 <- names(item_stability_proportions_iter5[item_stability_proportions_iter5 < stability_threshold])

      print(paste("Number of stable items (>= ", stability_threshold, ") after Iteration 5:", length(stable_items_iter5)))
      print(paste("Number of unstable items (< ", stability_threshold, ") after Iteration 5:", length(unstable_items_iter5)))

      if (length(unstable_items_iter5) == 0) {
        print("Convergence achieved! All items are now stable after 5 iterations.")
      } else {
        warning("Items STILL remain unstable after 5 iterations. The process should be stopped.")
      }
    }
  }
}

```

```

    }

    # Os itens estáveis desta rodada se tornam o conjunto de dados final
    if (length(stable_items_iter5) > 0) {
      final_stable_data <- data_after_uva_iter5[, stable_items_iter5, drop = FALSE]
      print("Dimensions of final stable data after Iteration 5:")
      print(dim(final_stable_data))

      # Roda a EGA final sobre o conjunto final de dados
      if (ncol(final_stable_data) >= 2) {
        print("Running final EGA on the final set of stable items...")
        final_ega_run <- EGA(data = final_stable_data, model = "glasso", verbose = FALSE)
      } else {
        final_ega_run <- list(n.dim = 0, wc = NULL)
      }
    } else {
      print("No stable items found after Iteration 5. Final dataset is empty.")
      final_stable_data <- data.frame()
      final_ega_run <- list(n.dim = 0, wc = NULL)
    }
  } else {
    print("bootEGA Iteration 5 was skipped as there were too few items after UVA.")
    final_stable_data <- data_after_uva_iter5
    if(ncol(final_stable_data) > 0){
      final_ega_run <- list(n.dim = 1, wc = NULL) # Tem 1 item, logo 1 dimensão
    } else {
      final_ega_run <- list(n.dim = 0, wc = NULL)
    }
  }
}
}

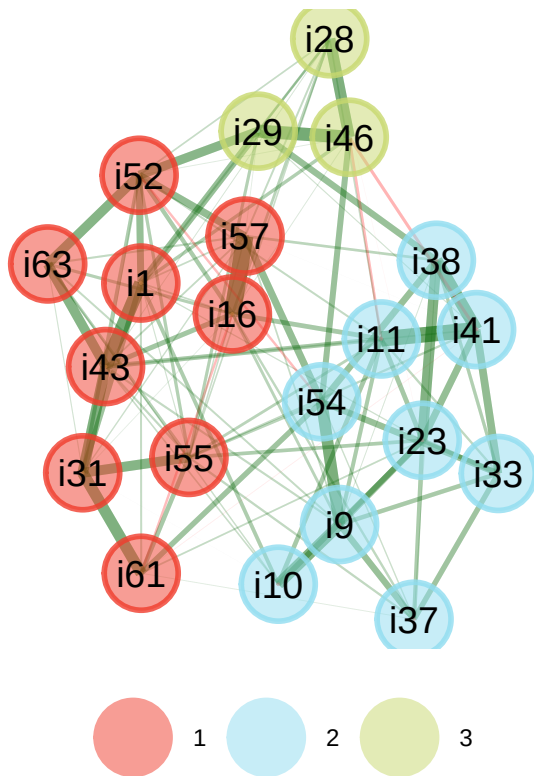
```

```

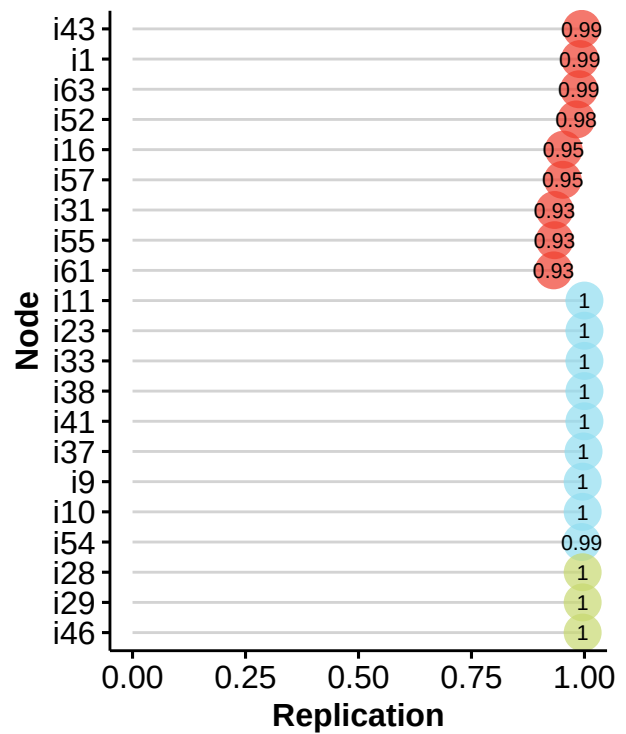
## [1] "--- Starting Iteration 5 of UVA and bootEGA ---"
## [1] "Dimensions of data for UVA Iteration 5 (current_data_iter5): 768 rows x 21 cols"
## [1] "Running UVA Iteration 5..."
## [1] "UVA Iteration 5 did not reduce items or resulted in empty data. Using input data."
## [1] "UVA Iteration 5 completed. Items remaining: 21"
## [1] "Running bootEGA Iteration 5..."

```

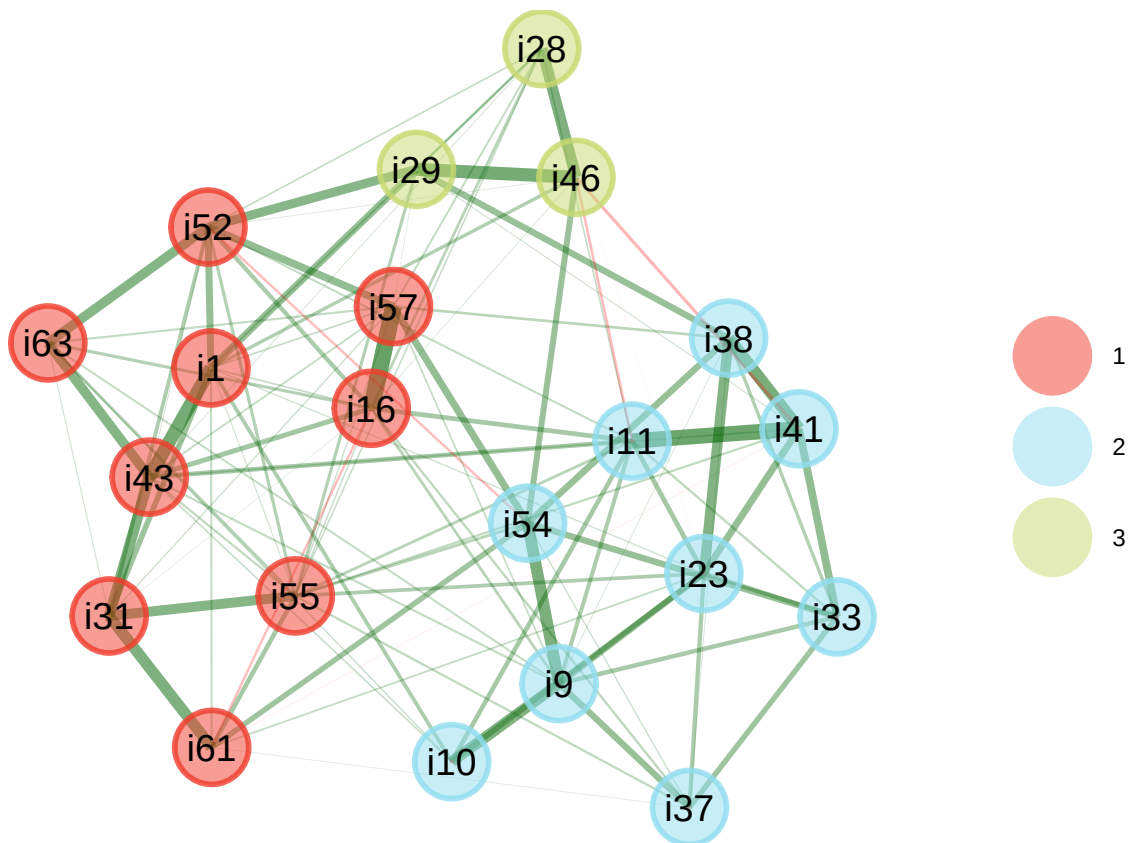
## Original Sample | EGA



Communities ● 1 ● 2 ● 3



```
## [1] "bootEGA Iteration 5 completed."
## [1] "Number of stable items (>= 0.75 ) after Iteration 5: 21"
## [1] "Number of unstable items (< 0.75 ) after Iteration 5: 0"
## [1] "Convergence achieved! All items are now stable after 5 iterations."
## [1] "Dimensions of final stable data after Iteration 5:"
## [1] 768 21
## [1] "Running final EGA on the final set of stable items..."
```



## 10 Análise “final”

Verificação dos itens por cluster:

```
if (!exists("final_stable_data")) {
  print("Error: 'final_stable_data' was not defined by the iterative process.")
  final_stable_data <- data.frame()
}
if (!exists("final_ega_run")) {
  print("Error: 'final_ega_run' was not defined by the iterative process.")
  final_ega_run <- list(n.dim = 0, wc = NULL)
}

# Definir n_final_items com base no estado ATUAL de final_stable_data
n_final_items <- if (!is.null(final_stable_data)) ncol(final_stable_data) else 0
print(paste("Final number of stable items for reporting:", n_final_items))
```

```
## [1] "Final number of stable items for reporting: 21"
```

```
if (n_final_items > 0 && !is.null(final_ega_run) && !is.null(final_ega_run$wc) && length(final_ega_run$wc) > 0) {
  print(paste("Final EGA identified", final_ega_run$n.dim, "dimensions (clusters).")) # Adicionado para o relatório
  final_cluster_assignments <- final_ega_run$wc
  final_stable_item_names <- names(final_cluster_assignments)
```

```

# Garantir que a função get_index_from_item_name está definida
if (!exists("get_index_from_item_name")) {
  get_index_from_item_name <- function(item_name) {
    num_str <- sub("^i", "", item_name)
    index <- suppressWarnings(as.integer(num_str))
    if (is.na(index)) {
      warning(paste("Could not parse index from item name:", item_name))
      return(NA)
    } else {
      return(index)
    }
  }
}

original_indices_in_unique_phrases <- sapply(final_stable_item_names,
                                             get_index_from_item_name,
                                             USE.NAMES = FALSE)

valid_indices_mask <- !is.na(original_indices_in_unique_phrases)
original_indices_in_unique_phrases_clean <- original_indices_in_unique_phrases[valid_indices_mask]
final_stable_item_names_clean <- final_stable_item_names[valid_indices_mask]
final_cluster_assignments_clean <- final_cluster_assignments[valid_indices_mask]

final_stable_phrases_text <- character(length(original_indices_in_unique_phrases_clean))
if (!is.null(unique_phrases) && length(unique_phrases) > 0) {
  within_bounds_mask <- original_indices_in_unique_phrases_clean >= 1 & original_indices_in_unique_phrases_clean <= length(unique_phrases)

  if(any(!within_bounds_mask)){
    warning("Some item indices were out of bounds for 'unique_phrases'. Corresponding phrases will be NA.")
  }
  final_stable_phrases_text[within_bounds_mask] <- unique_phrases[original_indices_in_unique_phrases_clean[within_bounds_mask]]
  final_stable_phrases_text[!within_bounds_mask] <- NA
} else {
  warning("'unique_phrases' is NULL or empty. All phrase texts will be NA.")
  final_stable_phrases_text[] <- NA
}

final_results_df <- data.frame(
  ItemName = final_stable_item_names_clean,
  OriginalIndex = original_indices_in_unique_phrases_clean,
  EstimatedCluster = final_cluster_assignments_clean,
  PhraseText = final_stable_phrases_text,
  stringsAsFactors = FALSE
)

final_results_df <- final_results_df[!is.na(final_results_df$PhraseText), ]

if (nrow(final_results_df) > 0) {
  final_results_df <- final_results_df[order(final_results_df$EstimatedCluster, final_results_df$OriginalIndex), ]

  print("--- Final Stable Phrases Grouped by Estimated EGA Cluster (with Item Numbers) ---")
  # Adicionado para plotar a rede final aqui, se desejado.
  if (!is.null(final_ega_run) && "EGA" %in% class(final_ega_run)) { # Verifica se é um objeto EGA

```

```

        plot(final_ega_run, title = "Final Network Structure")
    }

    for (cluster_id in unique(final_results_df$EstimatedCluster)) {
        cat("\n=== Cluster", cluster_id, "===\n")
        items_in_cluster <- final_results_df[final_results_df$EstimatedCluster == cluster_id, ]
        for (i in 1:nrow(items_in_cluster)) {
            cat(sprintf("%* %-5s %s\n",
                        items_in_cluster$ItemName[i],
                        items_in_cluster$PhraseText[i]))
        }
    }
} else {
    print("No final results to display after mapping items to phrases or all phrases were NA.")
}
} else if (n_final_items == 1 && !is.null(final_stable_data) && ncol(final_stable_data) == 1) {
    # Caso especial: apenas 1 item final, EGA não rodou.
    print(paste("Processo finalizado com 1 item restante:", colnames(final_stable_data)))
    single_item_name <- colnames(final_stable_data)
    if (!exists("get_index_from_item_name")) { # Garante que a função existe
        get_index_from_item_name <- function(item_name) {
            num_str <- sub("^i", "", item_name)
            index <- suppressWarnings(as.integer(num_str))
            if (is.na(index)) { return(NA) } else { return(index) }
        }
    }
    single_item_index <- get_index_from_item_name(single_item_name)
    if (!is.na(single_item_index) && !is.null(unique_phrases) && single_item_index >= 1 && single_item_index <= length(unique_phrases)) {
        cat("\nTexto do item final:\n")
        cat(sprintf("%* %-5s %s\n", single_item_name, unique_phrases[single_item_index]))
    }
} else {
    print("Final analysis and reporting skipped: no stable items, EGA run not performed, or no cluster")
}

```

```

## [1] "Final EGA identified 3 dimensions (clusters)."
```

```

## [1] "--- Final Stable Phrases Grouped by Estimated EGA Cluster (with Item Numbers) ---"
```

```

##
```

```

## === Cluster 1 ===
```

```

## * i1    Consegue imitar gestos simples associados a músicas espontaneamente?
```

```

## * i16   Faz comando conforme a letra da música?
```

```

## * i31   Se familiariza com canções e letras simples?
```

```

## * i43   Toca melodias simples ensinadas em encontros passados (sistema de representação perceptiva)?
```

```

## * i52   Identifica ritmo musical? (Rock, samba)
```

```

## * i55   Canta músicas mais complexas (andamento rápido, que exige mais vocabulário)?
```

```

## * i57   Conhece o momento certo das entradas da música quando está cantando em um playback?
```

```

## * i61   Aprende palavras novas com facilidade?
```

```

## * i63   Recorda nome de música associando-a ao seu compositor?
```

```

##
```

```

## === Cluster 2 ===
```

```

## * i9    Reconhece a voz cantada masculina da voz feminina?
```

```

## * i10   Reconhece as 'vozes' dos personagens de desenho (ex: Mickey no tablet sem ver)?
```

```

## * i11   Apresenta habilidade de figura/fundo, identificando a mensagem primária na presença de sons
```

```
## * i23   Reconhece diferentes sons produzidos pela boca?
## * i33   O volume da voz da criança é apropriado?
## * i37   Articula bem o 'L' e o 'R' em encontros consonantais? (Planeta, Branco)
## * i38   Sustenta um som vocálico durante 7 segundos?
## * i41   Apresenta uma boa percepção em relação à duração do som?
## * i54   Fala ao microfone sem dificuldade?
##
## === Cluster 3 ===
## * i28   Demonstra planejamento da coreografia em diferentes espaços (sala para palco)?
## * i29   Marcha e toca instrumentos simultaneamente sem perder o pulso?
## * i46   Consegue dançar livremente?

# Optional: Save the final results data frame
# if (exists("final_results_df") && nrow(final_results_df) > 0) {
#   write.csv(final_results_df, "final_stable_items_and_clusters_EGA_only.csv", row.names = FALSE)
# }
```

Agora conseguimos verificar, tanto visualmente quanto semanticamente, que os clusters estão mais coesos. Por tanto, nomeamos o cluster 1 como

- Cluster 1: Produção Musical
- Cluster 2: Percepção Musical
- Cluster 3: Corporeidade em música

## 11 Análise de centralidade

Os índices de centralidade da rede foram estimados utilizando o método EBICglasso para identificar os nós (itens) mais influentes na estrutura da rede. As seguintes medidas foram calculadas:

- Força (Strength): Indica o quão conectado diretamente um nó está aos outros na rede. Em uma rede de itens, um item com alta força tem muitas conexões fortes (correlações parciais) com outros itens, sugerindo que ele compartilha muitas características ou está fortemente relacionado a diversos aspectos do construto medido.
- Proximidade (Closeness): Mede a quão “próximo” um nó está, em média, de todos os outros nós na rede, considerando os caminhos mais curtos. Um item com alta proximidade pode rapidamente interagir ou influenciar outros itens na rede, sendo central em termos de eficiência de informação.
- Intermediação (Betweenness): Quantifica a frequência com que um nó aparece nos caminhos mais curtos entre outros pares de nós. Itens com alta intermediação atuam como “pontes” ou conectores entre diferentes subgrupos de itens ou aspectos do construto, sendo cruciais para a coesão da rede.
- Influência Esperada (Expected Influence - EI): Uma medida de centralidade específica para redes psicológicas que considera a soma das conexões ponderadas de um nó, onde conexões positivas e negativas podem contribuir. Em redes estimadas com EBICglasso (que geralmente são não direcionadas e onde as arestas representam correlações parciais condicionais), a EI de 1 passo é equivalente à Força. A EI de 2 passos considera também a influência indireta através de vizinhos.

Escores de centralidade padronizados (escores-z) foram então visualizados para facilitar a comparação e identificação dos itens mais proeminentes em cada medida de centralidade.

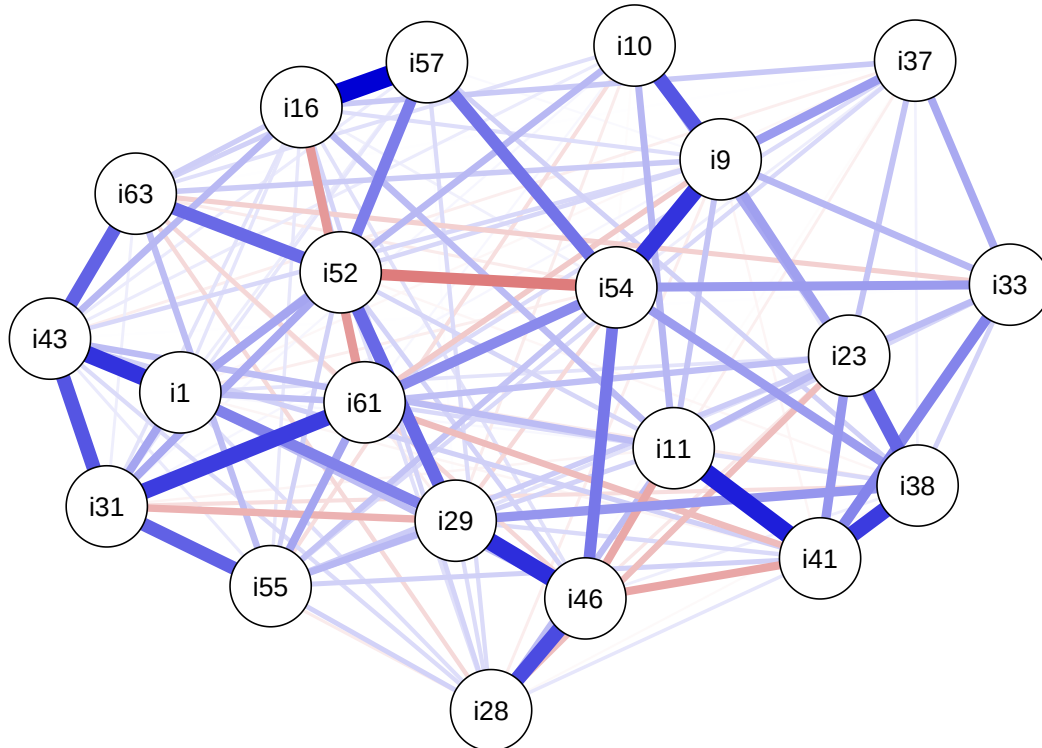
```

library("qgraph")
library("bootnet")

Network <- estimateNetwork(final_stable_data,
                           default = "EBICglasso", weighted = TRUE, tuning = 0.5)

plot(Network)

```

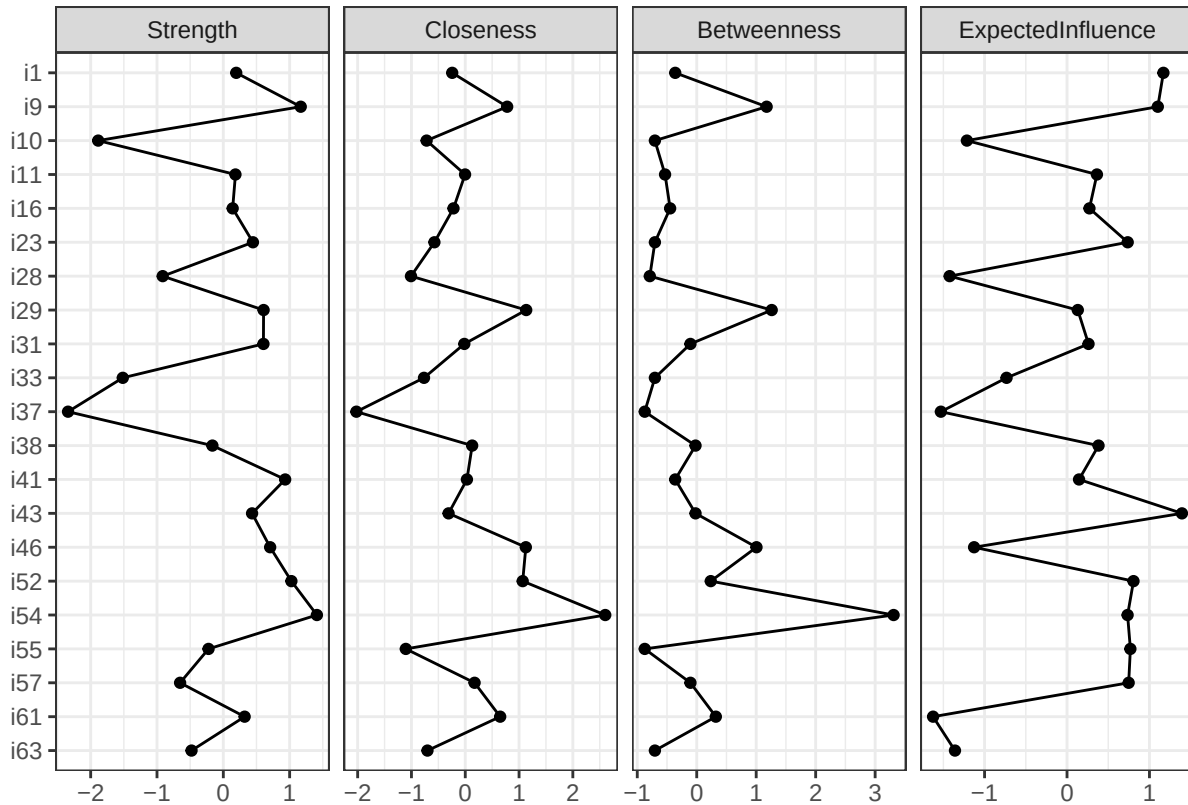


```

centralityPlot(Network, scale = c("z-scores"),
               include = c("Strength", "Closeness", "Betweenness", "ExpectedInfluence"),
               theme_bw = TRUE, print = TRUE,
               verbose = TRUE, weighted = TRUE,
               decreasing = T)

```





```
print(paste("Centralidade de força:", unique_phrases[54]))
```

```
## [1] "Centralidade de força: Fala ao microfone sem dificuldade?"
```

```
print(paste("Centralidade de Proximidade:", unique_phrases[54]))
```

```
## [1] "Centralidade de Proximidade: Fala ao microfone sem dificuldade?"
```

```
print(paste("Centralidade de Intermediação:", unique_phrases[54]))
```

```
## [1] "Centralidade de Intermediação: Fala ao microfone sem dificuldade?"
```

```
print(paste("Influência esperada:", unique_phrases[43]))
```

```
## [1] "Influência esperada: Toca melodias simples ensinadas em encontros passados (sistema de represen
```

## 12 Medidas de centralidade e estabilidade

A análise de estabilidade das medidas de centralidade, por meio do coeficiente de estabilidade de correlação (CS-coefficient), revelou que 'Influência Esperada' (CS = 0.67) bem como 'Força', 'Proximidade' e 'Intermediação' (CS = 0.0.439) foram métricas estáveis.

O item *Fala ao microfone sem dificuldade?* destacou-se como o mais central em termos de Força, intermediação e proximidade, indicando que esta habilidade possui o maior número de conexões diretas e fortes com as demais habilidades na rede, Também foi o item que teve maior Proximidade quanto em Intermediação, sugerindo que a habilidade de se expressar verbal/oralmente pode atuar como um importante conector e facilitador da interação entre diferentes conjuntos de habilidades na rede

Por fim, o item *Toca melodias simples ensinadas em encontros passados (sistema de representação perceptiva)?* apresentou a maior Influência Esperada, sugerindo que a capacidade de executar melodias simples ensinadas anteriormente é particularmente influente na estrutura geral das habilidades musicais avaliadas.

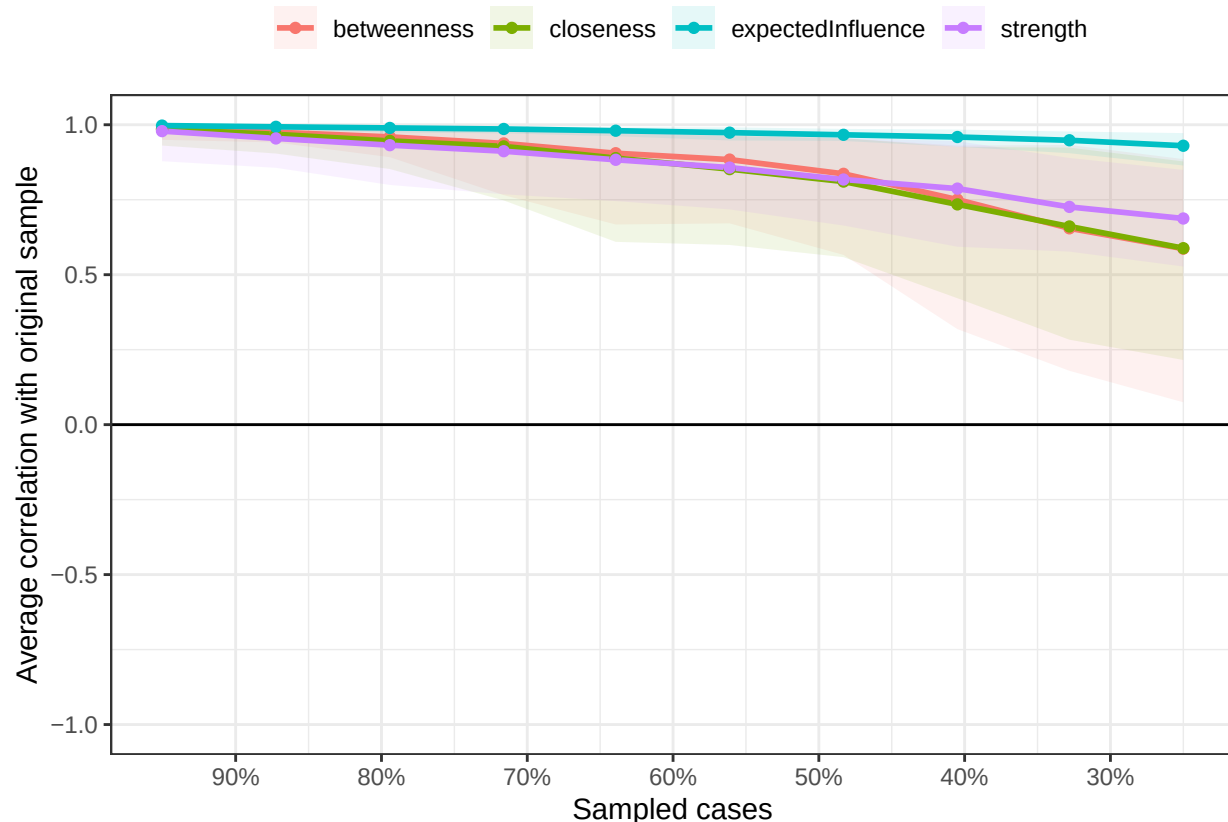
```
#Estabilidade da centralidade
```

```
boot <- bootnet(Network, nBoots = 1000, type = "case",
  statistics = c("strength", "closeness", "betweenness", "expectedInfluence"))
```

```
## |
```

```
## |
```

```
plot(boot, statistics= c("strength", "closeness", "betweenness", "expectedInfluence"))
```



```
#CS-coefficient
```

```
corStability(boot)
```

```
## === Correlation Stability Analysis ===
```

```
##
## Sampling levels tested:
##      nPerson Drop%   n
## 1      192  75.0   89
## 2      252  67.2  100
## 3      311  59.5   91
## 4      371  51.7  109
## 5      431  43.9   94
## 6      491  36.1  102
## 7      550  28.4  106
## 8      610  20.6  108
## 9      670  12.8  104
## 10     730   4.9   97
##
## Maximum drop proportions to retain correlation of 0.7 in at least 95% of the samples:
##
## betweenness: 0.439
##   - For more accuracy, run bootnet(..., caseMin = 0.361, caseMax = 0.517)
##
## closeness: 0.361
##   - For more accuracy, run bootnet(..., caseMin = 0.284, caseMax = 0.439)
##
## expectedInfluence: 0.75 (CS-coefficient is highest level tested)
##   - For more accuracy, run bootnet(..., caseMin = 0.672, caseMax = 1)
##
## strength: 0.439
##   - For more accuracy, run bootnet(..., caseMin = 0.361, caseMax = 0.517)
##
## Accuracy can also be increased by increasing both 'nBoots' and 'caseN'.
```

## 13 Referências

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers) (pp. 4171–4186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423> (Também disponível no arXiv: arXiv:1810.04805)

Russell-Lasalandra, L. L., Christensen, A. P., & Golino, H. (2024, September 12). Generative Psychometrics via AI-GENIE: Automatic Item Generation and Validation via Network-Integrated Evaluation. <https://doi.org/10.31234/osf.io/fgbj4>

Verga Pinto, A. B. (2024). Construcción de una herramienta multisistémica de evaluación en musicoterapia para el rango etario de 0 a 12 años: Manual del PREVIMMUS y su enfoque teórico [Trabajo de Fin de Máster, Universidad de Sevilla].

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP) (pp. 3982–3992). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>

```
## R version 4.5.0 (2025-04-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26100)
```

```

##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Portuguese_Brazil.utf8  LC_CTYPE=Portuguese_Brazil.utf8
## [3] LC_MONETARY=Portuguese_Brazil.utf8 LC_NUMERIC=C
## [5] LC_TIME=Portuguese_Brazil.utf8
##
## time zone: America/Sao_Paulo
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods    base
##
## other attached packages:
## [1] bootnet_1.6      ggplot2_3.5.2    qgraph_1.9.8     psych_2.5.3
## [5] psychTools_2.5.3 EGAnet_2.3.0      reticulate_1.42.0
##
## loaded via a namespace (and not attached):
##   [1] RColorBrewer_1.1-3      rstudioapi_0.17.1      jsonlite_2.0.0
##   [4] shape_1.4.6.1          magrittr_2.0.3         jomo_2.7-6
##   [7] farver_2.1.2           nloptr_2.2.1           rmarkdown_2.29
##  [10] vctrs_0.6.5            minqa_1.2.8            base64enc_0.1-3
##  [13] rstatix_0.7.2          htmltools_0.5.8.1      progress_1.2.3
##  [16] polynom_1.4-1          plotrix_3.8-4           weights_1.0.4
##  [19] broom_1.0.8            Formula_1.2-5           mitml_0.4-5
##  [22] parallelly_1.44.0      htmlwidgets_1.6.4      plyr_1.8.9
##  [25] igraph_2.1.4           lifecycle_1.0.4        iterators_1.0.14
##  [28] pkgconfig_2.0.3        Matrix_1.7-3           R6_2.6.1
##  [31] fastmap_1.2.0          rbibutils_2.3          future_1.40.0
##  [34] digest_0.6.37          fdrtool_1.2.18         colorspace_2.1-1
##  [37] GGally_2.2.1           ellipse_0.5.0          Hmisc_5.2-3
##  [40] ggpubr_0.6.0           labeling_0.4.3         progressr_0.15.1
##  [43] nnls_1.6              gdata_3.0.1            IsingSampler_0.2.3
##  [46] abind_1.4-8            compiler_4.5.0         proxy_0.4-27
##  [49] withr_3.0.2            doParallel_1.0.17      glasso_1.11
##  [52] htmlTable_2.4.3        backports_1.5.0        carData_3.0-5
##  [55] mgm_1.2-15            ggstats_0.9.0          R.utils_2.13.0
##  [58] ggsignif_0.6.4         pan_1.9                MASS_7.3-65
##  [61] corpcor_1.6.10         rtf_0.4-14.1           gtools_3.9.5
##  [64] tools_4.5.0           pbivnorm_0.6.0         foreign_0.8-90
##  [67] future.apply_1.11.3    nnet_7.3-20            R.oo_1.27.1
##  [70] glue_1.8.0            quadprog_1.5-8         NetworkToolbox_1.4.2
##  [73] nlme_3.1-168          grid_4.5.0            checkmate_2.3.2
##  [76] cluster_2.1.8.1       reshape2_1.4.4         generics_0.1.3
##  [79] snow_0.4-4            gtable_0.3.6          class_7.3-23
##  [82] R.methodsS3_1.8.2     tidyr_1.3.1           sna_2.8
##  [85] data.table_1.17.0     hms_1.1.3             car_3.1-3
##  [88] foreach_1.5.2         pillar_1.10.2          stringr_1.5.1
##  [91] splines_4.5.0         dplyr_1.1.4           smacof_2.1-7
##  [94] networktools_1.6.0    lattice_0.22-7         survival_3.8-3
##  [97] tidyselect_1.2.1      pbapply_1.7-2          knitr_1.50
## [100] reformulas_0.4.1      gridExtra_2.3          IsingFit_0.4

```

|          |                       |                  |                   |
|----------|-----------------------|------------------|-------------------|
| ## [103] | stats4_4.5.0          | xfun_0.52        | stringi_1.8.7     |
| ## [106] | statnet.common_4.11.0 | yaml_2.3.10      | boot_1.3-31       |
| ## [109] | evaluate_1.0.3        | codetools_0.2-20 | wordcloud_2.6     |
| ## [112] | tibble_3.2.1          | cli_3.6.5        | rpart_4.1.24      |
| ## [115] | Rdpack_2.6.4          | lavaan_0.6-19    | network_1.19.0    |
| ## [118] | Rcpp_1.0.14           | globals_0.18.0   | coda_0.19-4.1     |
| ## [121] | png_0.1-8             | parallel_4.5.0   | prettyunits_1.2.0 |
| ## [124] | jpeg_0.1-11           | lme4_1.1-37      | listenv_0.9.1     |
| ## [127] | glmnet_4.1-8          | mvtnorm_1.3-3    | e1071_1.7-16      |
| ## [130] | scales_1.4.0          | eigenmodel_1.11  | purrr_1.0.4       |
| ## [133] | crayon_1.5.3          | rlang_1.1.6      | cowplot_1.1.3     |
| ## [136] | mnormt_2.1.1          | mice_3.17.0      |                   |